МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №4

з дисципліни «Функціональне програмування»

на тему: «Рекурсія та мемоізація в Python»

Варіант 6: «Підрахунок суми вкладеної структури»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Дослідити принципи рекурсивних обчислень, базові випадки та рекурсивні кроки; розібратися з лімітом стеку викликів у Python (sys.getrecursionlimit()) та відсутністю TCO; застосувати functools.lru_cache для мемоізації; реалізувати рекурсивний підрахунок суми та кількості вкладених числових елементів, максимальну глибину та безпечну ітеративну версію з власним стеком.

2. Теоретичні відомості

Рекурсія: спосіб організації обчислень, за якого функція викликає саму себе для вирішення підзадачі меншого розміру. Обов'язково містить базовий випадок (вихід з рекурсії) та рекурсивний крок.

Стек викликів CPython: кожен виклик створює новий стек-фрейм. За замовчуванням ліміт глибини рекурсії становить 1000. Перевищення ліміту породжує виняток RecursionError. У стандартному Python оптимізація хвостової рекурсії (TCO) відсутня.

Мемоізація (lru_cache): техніка оптимізації, що кешує результати чистих функцій за значеннями їхніх аргументів. Дозволяє знизити експоненціальну складність O(2^n) до лінійної O(n).

Ітеративний обхід зі стеком: зберігає стан у користувацькій структурі даних (списку) в динамічній пам'яті (heap), усуваючи обмеження системного стеку викликів.

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Підрахунок суми вкладеної структури.

Вхідні дані: Довільно вкладені list та tuple із числовими значеннями.

Основні задачі: Рекурсивно обчислити суму (nested_sum) та кількість (nested_count) елементів, обчислити спільну статистику (nested_stats).

Додаткове дослідження: Максимальна глибина вкладеності (max_depth), ітеративна версія зі стеком (iterative_nested_stats), поведінка на глибині понад 1000 елементів.

Базові алгоритми: quicksort_pure, quicksort_hybrid, обходи бінарного дерева, мемоізований Фібоначчі.

4. Хід роботи та лістинг програми

Модуль вкладених структур (варіант 6) (nested.py)

```
from __future__ import annotations

from typing import Any


def nested_sum(data: Any) -> float:
    if isinstance(data, (int, float)) and not isinstance(data, bool):
        return float(data)
    if isinstance(data, (list, tuple)):
        return float(sum(nested_sum(item) for item in data))
    return 0.0


def nested_count(data: Any) -> int:
    if isinstance(data, (int, float)) and not isinstance(data, bool):
        return 1
    if isinstance(data, (list, tuple)):
        return sum(nested_count(item) for item in data)
    return 0


def nested_stats(data: Any) -> tuple[float, int]:
    if isinstance(data, (int, float)) and not isinstance(data, bool):
        return float(data), 1
    if isinstance(data, (list, tuple)):
        total_sum = 0.0
        total_count = 0
        for item in data:
            s, c = nested_stats(item)
            total_sum += s
            total_count += c
        return total_sum, total_count
    return 0.0, 0


def max_depth(data: Any) -> int:
    if not isinstance(data, (list, tuple)):
        return 0
    if not data:
        return 1
    return 1 + max(max_depth(item) for item in data)


def iterative_nested_stats(data: Any) -> tuple[float, int]:
    stack: list[Any] = [data]
    total_sum = 0.0
    count = 0

    while stack:
        curr = stack.pop()
        if isinstance(curr, (int, float)) and not isinstance(curr, bool):
            total_sum += float(curr)
```

```
            count += 1
        elif isinstance(curr, (list, tuple)):
            for item in reversed(curr):
                stack.append(item)

    return total_sum, count


def iterative_max_depth(data: Any) -> int:
    if not isinstance(data, (list, tuple)):
        return 0
    if not data:
        return 1

    stack: list[tuple[Any, int]] = [(data, 1)]
    max_d = 1

    while stack:
        curr, d = stack.pop()
        if d > max_d:
            max_d = d
        if isinstance(curr, (list, tuple)):
            for item in curr:
                if isinstance(item, (list, tuple)):
                    stack.append((item, d + 1))

    return max_d
```

Базові алгоритми та мемоізація (algorithms.py)

```
from __future__ import annotations

from dataclasses import dataclass
from functools import cache
from typing import Generic, TypeVar

T = TypeVar("T")


def insertion_sort(seq: list[int]) -> list[int]:
    result = list(seq)
    for i in range(1, len(result)):
        key = result[i]
        j = i - 1
        while j >= 0 and result[j] > key:
            result[j + 1] = result[j]
            j -= 1
        result[j + 1] = key
    return result


def quicksort_pure(seq: list[int]) -> list[int]:
    if len(seq) <= 1:
        return list(seq)
    pivot = seq[len(seq) // 2]
    left = [x for x in seq if x < pivot]
    mid = [x for x in seq if x == pivot]
    right = [x for x in seq if x > pivot]
    return quicksort_pure(left) + mid + quicksort_pure(right)
```

```
def quicksort_hybrid(seq: list[int], threshold: int = 16) -> list[int]:
    if len(seq) <= 1:
        return list(seq)
    if len(seq) <= threshold:
        return insertion_sort(seq)
    pivot = seq[len(seq) // 2]
    left = [x for x in seq if x < pivot]
    mid = [x for x in seq if x == pivot]
    right = [x for x in seq if x > pivot]
    return quicksort_hybrid(left, threshold) + mid + quicksort_hybrid(right, threshold)


@dataclass(frozen=True, slots=True)
class TreeNode(Generic[T]):
    val: T
    left: TreeNode[T] | None = None
    right: TreeNode[T] | None = None


def preorder(node: TreeNode[T] | None) -> list[T]:
    if node is None:
        return []
    return [node.val] + preorder(node.left) + preorder(node.right)


def inorder(node: TreeNode[T] | None) -> list[T]:
    if node is None:
        return []
    return inorder(node.left) + [node.val] + inorder(node.right)


def postorder(node: TreeNode[T] | None) -> list[T]:
    if node is None:
        return []
    return postorder(node.left) + postorder(node.right) + [node.val]


def fib_naive(n: int) -> int:
    if n < 0:
        raise ValueError("n must be >= 0")
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)


@cache
def fib_memoized(n: int) -> int:
    if n < 0:
        raise ValueError("n must be >= 0")
    if n <= 1:
        return n
    return fib_memoized(n - 1) + fib_memoized(n - 2)
```

Демонстраційний модуль (main.py)

```
from __future__ import annotations
```

```
import sys
import time

from algorithms import fib_memoized, fib_naive, quicksort_hybrid
from nested import (
    iterative_max_depth,
    iterative_nested_stats,
    max_depth,
    nested_count,
    nested_stats,
    nested_sum,
)


def main() -> None:
    data = [1, [2, 3], (4, [5, (6, 7)]), [8, [9, [10]]]]

    print("Вхідна вкладена структура:", data)
    print("nested_sum:", nested_sum(data))
    print("nested_count:", nested_count(data))
    s, c = nested_stats(data)
    print(f"nested_stats (рекурсивно): сума={s}, к-сть={c}")

    it_s, it_c = iterative_nested_stats(data)
    print(f"iterative_nested_stats:   сума={it_s}, к-сть={it_c}")

    print("max_depth (рекурсивно):", max_depth(data))
    print("iterative_max_depth:   ", iterative_max_depth(data))

    fib_memoized.cache_clear()
    t0 = time.perf_counter()
    res_m = fib_memoized(32)
    tm = time.perf_counter() - t0
    info = fib_memoized.cache_info()
    print(f"\nfib_memoized(32) = {res_m} за {tm:.6f}s | CacheInfo: {info}")

    t0 = time.perf_counter()
    res_n = fib_naive(32)
    tn = time.perf_counter() - t0
    print(f"fib_naive(32)    = {res_n} за {tn:.6f}s (прискорення: {tn / tm:.1f}x)")

    sample_list = [5, 2, 8, 1, 9, 3, 7, 4, 6]
    print("\nquicksort_hybrid:", quicksort_hybrid(sample_list))
    print("Поточний ліміт стеку Python:", sys.getrecursionlimit())


if __name__ == "__main__":
    main()
```

Тести вкладених структур (tests/test_nested.py)

```
from typing import Any

import pytest
from nested import (
    iterative_max_depth,
    iterative_nested_stats,
    max_depth,
    nested_count,
```

```
    nested_stats,
    nested_sum,
)


def test_nested_sum_basic(nested_sample: list[Any]) -> None:
    assert nested_sum(nested_sample) == 55.0


def test_nested_count_basic(nested_sample: list[Any]) -> None:
    assert nested_count(nested_sample) == 10


def test_nested_stats(nested_sample: list[Any]) -> None:
    s, c = nested_stats(nested_sample)
    assert s == 55.0
    assert c == 10


def test_iterative_equivalence(nested_sample: list[Any]) -> None:
    rec_s, rec_c = nested_stats(nested_sample)
    it_s, it_c = iterative_nested_stats(nested_sample)
    assert rec_s == it_s
    assert rec_c == it_c


def test_max_depth(nested_sample: list[Any]) -> None:
    assert max_depth(nested_sample) == 4
    assert iterative_max_depth(nested_sample) == 4


def test_edge_cases() -> None:
    assert nested_sum([]) == 0.0
    assert nested_count([]) == 0
    assert max_depth([]) == 1
    assert nested_sum(42) == 42.0
    assert nested_count(42) == 1
    assert max_depth(42) == 0
    assert nested_sum([True, False, 10]) == 10.0


def test_deeply_nested_structure() -> None:
    depth = 1200
    curr: Any = 1
    for _ in range(depth):
        curr = [curr]

    it_s, it_c = iterative_nested_stats(curr)
    assert it_s == 1.0
    assert it_c == 1
    assert iterative_max_depth(curr) == depth

    with pytest.raises(RecursionError):
        nested_sum(curr)
```

Тести Фібоначчі та lru_cache (tests/test_fib.py)

```
import pytest
from algorithms import fib_memoized, fib_naive
```

```
def test_fib_correctness() -> None:
    expected = [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
    for i, exp in enumerate(expected):
        assert fib_naive(i) == exp
        assert fib_memoized(i) == exp


def test_fib_memoization_cache_info() -> None:
    fib_memoized.cache_clear()
    _ = fib_memoized(10)
    info = fib_memoized.cache_info()
    assert info.hits > 0
    assert info.currsize > 0


def test_fib_negative() -> None:
    with pytest.raises(ValueError):
        fib_naive(-1)
    with pytest.raises(ValueError):
        fib_memoized(-1)
```

Тести сортування quicksort (tests/test_quicksort.py)

```
from algorithms import insertion_sort, quicksort_hybrid, quicksort_pure


def test_quicksort_pure() -> None:
    data = [9, 3, 7, 1, 8, 2, 5, 4, 6]
    assert quicksort_pure(data) == [1, 2, 3, 4, 5, 6, 7, 8, 9]
    assert quicksort_pure([]) == []
    assert quicksort_pure([42]) == [42]


def test_insertion_sort() -> None:
    data = [5, 1, 4, 2, 8]
    assert insertion_sort(data) == [1, 2, 4, 5, 8]


def test_quicksort_hybrid() -> None:
    data = [10 - i for i in range(30)]
    assert quicksort_hybrid(data, threshold=8) == sorted(data)
```

Тести обходів дерева (tests/test_tree.py)

```
from algorithms import TreeNode, inorder, postorder, preorder


def test_tree_traversals() -> None:
    tree = TreeNode(
        val=4,
        left=TreeNode(val=2, left=TreeNode(1), right=TreeNode(3)),
        right=TreeNode(val=6, left=TreeNode(5), right=TreeNode(7)),
    )
    assert preorder(tree) == [4, 2, 1, 3, 6, 5, 7]
    assert inorder(tree) == [1, 2, 3, 4, 5, 6, 7]
```

```
    assert postorder(tree) == [1, 3, 2, 5, 7, 6, 4]


def test_empty_tree() -> None:
    assert preorder(None) == []
    assert inorder(None) == []
    assert postorder(None) == []
```

5. Результати виконання та тестування

Запуск демонстраційного модуля main.py

```
$ python main.py
Вхідна вкладена структура: [1, [2, 3], (4, [5, (6, 7)]), [8, [9, [10]]]]
nested_sum: 55.0
nested_count: 10
nested_stats (рекурсивно): сума=55.0, к-сть=10
iterative_nested_stats:   сума=55.0, к-сть=10
max_depth (рекурсивно): 4
iterative_max_depth:    4

fib_memoized(32) = 2178309 за 0.000024s | CacheInfo: CacheInfo(hits=30, misses=33, maxsize=None, cu
rrsize=33)
fib_naive(32)    = 2178309 за 0.615902s (прискорення: 25241.9x)

quicksort_hybrid: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Поточний ліміт стеку Python: 1000
```

Результати виконання тестів pytest

```
$ pytest -q
...............                                                          [100%]
15 passed in 0.59s
```

Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 8 source files
```

Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
8 files would be left unchanged.
```

6. Висновки

У лабораторній роботі №4 реалізовано рекурсивні та ітеративні алгоритми обробки складних вкладених структур. Продемонстровано роботу кешу lru_cache з колосальним прискоренням (понад 25 000 разів). Досліджено обмеження стеку викликів CPython: на

глибині 1200 рекурсивна версія очікувано викликає RecursionError, тоді як ітеративна версія з власним стеком успішно виконує обчислення без збоїв.